

Hybrid Mechanical and Agentic Specification Inference for the Move Prover

Wolfgang Grieskamp* and Teng Zhang and Vineeth Kashyap
Aptos Labs, Palo Alto, USA

*Corresponding author: wg@aptoslabs.com

Abstract—In this paper, we describe early work on a *specification inference tool* for the Move Prover that combines a *weakest-precondition (WP) analysis* over Move bytecode with an *agentic coding CLI* such as Claude Code. Specification inference reduces the boilerplate of writing specifications in Move: in order to verify a high-level property such as a global state invariant, pre- and post-conditions for the supporting functions typically have to be written by hand, which is tedious. In our setting, a Model Context Protocol (MCP) service exposes the WP analysis and the prover itself to the coding agent. The WP analysis provides a sound, mechanical baseline for inference; the AI is used precisely where WP is weakest — for loop invariants and high-level idiomatic specifications such as monotonicity, conservation, and structural invariants. The Move Prover serves as the oracle that decides whether the generated specs are valid, and the agent is equipped to generate proof hints and to refine the inferred specification until verification succeeds. The tool has been applied to a corpus of canonical Move code, including code that uses higher-order functions, dynamic dispatch, global state, references, and various forms of loops.

I. INTRODUCTION

The Move Prover (MVP) [1] is a formal verification tool for Move smart contracts [2, 3], designed for routine use during code development, with running times comparable to type checkers and linters. MVP is deployed at Aptos [4, 5] for formal verification of core protocol logic such as staking, metering, code deployment, and auxiliary data structures that underpin the framework.

A practical bottleneck in program verification is the cost of *writing* the specifications that drive verification. The properties developers want to establish are typically high-level: that a call chain aborts only under stated preconditions, that a global storage invariant is preserved, that funds are conserved across a transfer. While MVP can verify unspecified callees by inlining their bodies, this does not scale: modular verification requires explicit per-function pre- and postcondition specifications. This detail is mechanical and tedious to write, dominates the cost of using a verifier, and is a significant barrier to wider adoption.

Specification inference is a promising approach. Inference has a long history [6] and has gained steam again with LLMs [7, 8]. Coding agents like Claude Code [9] open further avenues: they expose tools to the AI (e.g. via the Model Context Protocol, MCP) and provide a workflow in which AI and developers collaborate on code changes. And models are significantly more powerful than three years ago.

We present early work on a novel combination of (a)

mechanical spec inference via weakest-precondition analysis (WP) [10, 11] and (b) agentic inference guided by a set of skills (English instructions), integrated as a plugin into Claude Code [12]. Our goal is a methodology in which AI, mechanical deduction, and users collaborate on writing and verifying Move specifications. This paper defines some of the building blocks toward that goal, focusing on spec inference.

By reusing MVP’s reference-elimination pass [1] (also adopted by Aeneas [13]), WP propagates over a reference-free intermediate form on which classical Dijkstra-style reasoning applies directly, and recovers abort conditions, modifies clauses, and consequence-style postconditions from the bytecode.

However, the principal bottleneck of WP-based inference is *loop invariants*: backward propagation through a loop requires an inductive invariant that is in general not derivable from the loop body alone, and without it the rest of the inferred specification becomes vacuous. LLMs help here: they are good at the pattern recognition loop invariants demand — spotting that a counter is monotone, that a sum is preserved, that an element stays in a sorted prefix — and at proposing specifications in the idiom a human would use.

Both signals are validated against the code by the prover; counter-examples feed back into the agent for refinement, and the user can intervene at any point.

Contributions. While primarily a tools paper with preliminary results, this work presents several novel ideas. As far as we know, previous work on LLM-based inference did not combine this approach with a mechanical component like WP. Earlier inference work is also not integrated with an interactive coding agent that lets the user contribute. Moreover, our WP algorithm benefits from reference elimination, as does our verification engine; this combination has not been described before, and the same approach extends to comparable verification engines for safe Rust (e.g. [13]).

Outline. Sec. II starts with a step-by-step example illustrating how the tool operates for the end user. Sec. III outlines WP analysis for Move code in the presence of Rust-style references and their elimination by the Move prover via bytecode rewriting. In Sec. IV we discuss the system of skills and prompts we have set up for inference and verification. Sec. V concludes with a discussion of related and future work.

II. STEP-BY-STEP EXAMPLE

We illustrate the workflow through a number of examples; both they and the tool are publicly available [14, 15].

A. Higher-Order Vector Search

We start with a simple function that finds an element in a vector matching a predicate:

```
fun find<T>(v: &vector<T>,
           pred: |&T|bool has copy + drop): u64 {
    let i = 0;
    let n = v.length();
    while (i < n) {
        if (pred(&v[i])) return i;
        i = i + 1;
    }
    n
}
```

We ask Claude Code (CC) to infer specs for this function; this loads a skill that creates a list of tasks:

> infer specs for the functions in this package

- Skill(move-flow:move-inf)
 - └ Successfully loaded skill
- * Zigzagging...
 - └ ☐ Synthesize loop invariants
 - ☐ Infer weakest preconditions
 - ☐ Simplify inferred specs
 - ☐ Full-scope verification run
 - ☐ Fix logical errors
 - ☐ Resolve timeouts

The tasks are listed in the suggested execution order but operate on independent snapshots of the package; CC may interleave them with edits that the user can monitor or override.

Synthesizing Loop Invariants. For this task, CC infers loop invariants for loops that do not yet have any. The skill defines Move-specific guidelines for this. For find, the resulting code is:

```
public fun find<T>(..)
..
while (i < n) {
    if (pred(vector::borrow(v, i))) return i;
    i = i + 1;
} spec {
    invariant i ≤ n;
    invariant n = len(v);
    invariant ∀ j in 0..i: !result_of<pred>(v[j]);
    invariant ∀ j in 0..i: !aborts_of<pred>(v[j]);
};
n
}
```

Notice the behavioral predicates `result_of` and `aborts_of` accessing the specification of another function — here the runtime value `pred`¹. Behavioral predicates are key for inference: we can refer to what other functions compute while abstracting away how, localizing inference to intra-function analysis.

Inferring Weakest Preconditions. For this task, CC calls MVP's WP algorithm via its MCP service, producing:

```
spec find {
    pragma opaque = true;
    ensures
        (∀ x: u64: x < len(v)
         ⇒ !result_of<pred>(v[x]))
        ⇒ result = len(v);
    ensures
        ∀ y: u64: (∀ x: u64:
                    x < y ⇒ !result_of<pred>(v[x]))
                    && (y < len(v) && result_of<pred>(v[y]))
                    ⇒ result = y;
    // aborts_if omitted
}
```

The `aborts_if` clauses have been omitted for brevity.

Simplification of Inferred Specs. WP derives semantically correct but not always ideal specifications. With loop invariants from CC and WP-derived specs from the Move Prover, we now ask CC to simplify the result.

```
spec find {
    pragma opaque = true;
    ensures result ≤ len(v);
    ensures ∀ j: u64 where j < result:
        !result_of<pred>(v[j]);
    ensures result < len(v)
        ⇒ result_of<pred>(v[result]);
    // aborts_if omitted
}
```

This specification is more compact than the WP baseline. In general, CC frequently rewrites WP-generated specs into more compact and readable form, despite significant term simplification already in the WP engine.

Verification. The remaining three tasks are verification-driven: a fast full-scope run surfaces logical errors, which are fixed iteratively; timeouts are then resolved by lemmas, proof hints, and helper functions (Sec. IV). In this example, verification immediately succeeds.

B. Exponentiation

Next, an example in the arithmetic domain.

```
fun pow(base: u64, exp: u64): u64 {
    let result = 1;
    let i = 0;
    while (i < exp) {
        result = result * base;
        i = i + 1;
    }
    result
}
```

In the first step, CC derives loop invariants, introducing a recursive helper function:

```
fun pow(base: u64, exp: u64): u64 {
    ..
    while (i < exp) {
        ..
    } spec {
        invariant i ≤ exp;
        invariant result = pow_spec(base, i);
    }
    result
}
spec fun pow_spec(base: u64, exp: u64): u64 {
    if (exp == 0) 1
    else base * pow_spec(base, exp - 1)
}
```

¹Specifications with function values are described in [16]

WP analysis then adds:

```
spec pow {
  ensures result = pow_spec(base, exp);
  aborts_if exp > 0
    && pow_spec(base, exp - 1) * base > MAX_U64;
}
```

CC-based simplification does not find anything to improve here. However, as is to be expected with non-linear arithmetic, verification times out.

Following the timeout-resolution instructions, CC introduces a monotonicity lemma via Move’s proof-hint system. The lemma is required for the `aborts_if` condition: if multiplication aborts with overflow earlier in the loop, then by monotonicity it also aborts later. Below, `num` is Move’s unbounded-precision signed integer type, available in specifications:

```
spec pow {
  .. // function spec above
} proof {
  // Apply lemma with trigger
  ∀ x: num, y: num
    {pow_spec(base, x), pow_spec(base, y)}
  apply pow_mono(base, x, y);
}
spec lemma pow_mono(base: num, x: num, y: num) {
  requires base ≥ 1 && 0 ≤ x && x ≤ y;
  ensures pow_spec(base, x) ≤ pow_spec(base, y);
} proof {
  if (x < y) {
    assert pow_spec(base, y - 1)
      ≤ pow_spec(base, y);
    apply pow_mono(base, x, y - 1);
  }
}
```

C. State-Dependent Inference

As a final example, we look at a function that modifies global storage at multiple addresses in sequence, illustrating state-dependent specifications.

```
struct Balance has key { v: u64 }
fun split_balance(
  dst1: &signer, dst2: &signer,
  src: address): u64 {
  let Balance { v } = move_from<Balance>(src);
  let half = v / 2;
  move_to(dst1, Balance { v: half });
  move_to(dst2, Balance { v: v - half });
  half
}
```

After running spec inference and simplification, we obtain:

```
spec split_balance(
  dst1: &signer, dst2: &signer,
  src: address): u64 {
  pragma opaque = true;
  modifies Balance[src];
  modifies Balance[signer::address_of(dst1)];
  modifies Balance[signer::address_of(dst2)];
  ensures result = old(Balance[src]).v / 2;
  ensures ..S1 |~ remove<Balance>(src);
  ensures S1..S2 |~
    publish<Balance>(signer::address_of(dst1),
      Balance{v: old(Balance[src]).
        v / 2});
  ensures S2.. |~
    publish<Balance>(signer::address_of(dst2),
```

```
Balance{v: old(Balance[src]).
  v
  - old(Balance[src]
    ).v / 2});
  aborts_if !∃<Balance>(src);
  aborts_if S1 |~
    ∃<Balance>(signer::address_of(dst1));
  aborts_if S2 |~
    ∃<Balance>(signer::address_of(dst2));
}
```

S1 and S2 are state labels naming intermediate memory states — here, the memory between the `move_from` and the first `move_to`, and between the two `move_tos`, respectively. The modality `..S1 |~ e, S1..S2 |~ e, S2.. |~ e` evaluates `e` over the named memory range, while `S |~ e` evaluates at the single state `S`. The global-memory operators are `publish<R>(addr, val)`, which asserts that resource `R` at `addr` equals `val`, `remove<R>(addr)`, which asserts that no resource of type `R` is present at `addr`, and `∃<R>(addr)`, the standard Move resource existence predicate.

III. WEAKEST-PRECONDITION ANALYSIS

We run a textbook WP analysis [10, 11] over the bytecode MVP hands to its Boogie back-end, and read off abort conditions, modifies clauses, and consequence-style postconditions. The novelty here is not in the WP algorithm, but in the *shape of the bytecode it operates on*: by the time the analysis runs, MVP’s standard transformation pipeline has already eliminated all Move references and injected every available specification as in-line `assume/assert` statements. The result is a reference-free, contract-annotated control flow on which backward predicate propagation reduces to the classical case.

Reference elimination. Move has Rust-style references: `&T` for shared reads and `&mut T` for exclusive writes, both with a stack-bounded lifetime. Predicate-transformer reasoning over such references is awkward, because the meaning of `*r` depends on which root location `r` was derived from. MVP avoids this by rewriting bytecode into a reference-free intermediate form [1]. Shared references are inlined: `&T` disappears and reads through it become reads of the underlying value. Mutable references are replaced by in/out parameters.

Specification instrumentation. Before WP runs, the bytecode passes through MVP’s regular specification instrumentation. Each function-level `requires`, `aborts_if`, `ensures`, or `modifies` clause, and each loop invariant, is compiled into an `assume` or `assert` at the appropriate program point. Calls are instrumented uniformly via the *behavioral predicates* of [16]: `requires_of<f>`, `aborts_of<f>`, `ensures_of<f>`, `modifies_of<f>`, and `result_of<f>` reify a callee’s contract as predicates over the call site’s arguments and result. Loops are broken at their headers via the standard *cut-point* encoding of Barnett and Leino for unstructured programs [11], with the loop invariant playing the role of cut-point predicate: the back-edge is severed, the invariant is asserted before and after each iteration, and the loop-modified locations are havoced in between, leaving an acyclic fragment through which WP propagates as through any straight-line code.

At the input to WP, the bytecode therefore looks like a sequence of pure assignments interleaved with `assume` and `assert` over plain values; the surface-level features (references, resources, calls, loops, user specifications) are gone.

IV. SKILLS, HOOKS, AND MCP INTEGRATION

The agentic side of the workflow is delivered as a Claude Code plugin, packaging skills (markdown instructions), edit-time hooks, and an MCP server. A text templating system lets shared content (e.g. the verification sub-workflow) be included by reference rather than duplicated. The setup is at [15]; we summarize the major design aspects.

Skills. The inference skill registers the workflow of Sec. II as a fixed task sequence — synthesize loop invariants, run WP, simplify, run full-scope verification, fix logical errors, resolve timeouts — and tags every emitted condition with an [inferred] marker so that re-runs are idempotent and never disturb user-written specs. Beyond the task script, the skills consolidate the domain guidance the agent would otherwise have to rediscover.

- *Loop invariants:* when to introduce recursive helper functions for iterative quantities, and when to prefer state-level invariants on resources — assumed by the prover at every call site, including each loop iteration — over per-loop inductive arguments.
- *Simplification:* discard vacuous conditions; replace unbounded quantifiers, a frequent source of SMT timeouts, with bounded or closed-form equivalents; and require trigger annotations on the surviving ones.
- *Timeout resolution:* a hierarchy of escalations — strengthen state-level invariants, then introduce spec helpers, then lemmas, then explicit proof scripts — with a strict prohibition on weakening abort or postconditions to silence a timeout, since this trades a real property for the appearance of progress.

Via template inclusion, the simplification and timeout-resolution material is shared with a standalone verification skill, so the same guidance applies whether the spec was inferred or written by hand.

Hooks. Event hooks couple the plugin to the agent’s life cycle. The most important hook is `PostToolUse`, triggered after every `Edit` or `Write` on a `.move` file: it parses the file, runs offline AST checks, and auto-formats the generated source. When any check fails, the hook exits non-zero and the diagnostic is surfaced into the agent’s transcript as immediate feedback for the next turn — in effect, a fast linter loop without the cost of a full prover invocation.

MCP integration. The plugin ships a studio MCP server that exposes the Move toolchain to CC as a curated set of tools, including MVP for WP analysis and verification. The MCP commands let the agent narrow calls to single functions or modules, with parameters such as timeout duration. The advantage of MCP over a CLI is a long-lived cache for the active package. Each tool call is then cheap relative to a fresh

run from source, making the iterative *infer / verify / refine* loop tractable at the fine granularity inference requires.

V. DISCUSSION AND CONCLUSION

A. Evaluation and Open Questions

We have applied the approach to examples from the basic Move libraries, covering loops, higher-order functions, arithmetic, state-dependent algorithms, and more. While the WP algorithm is stable and well-tested, the definition of skills and instructions is an ever-moving target.

Ordering of Inference Tasks. Currently the agent walks through the inference tasks — loop invariants, WP analysis, simplification, and verification — in a fixed order. We arrived at this order experimentally. In one experiment, WP first derived basic specs, then the model inferred invariants, and WP ran again. The vacuous specifications from the first run were unhelpful and led the model into unfavourable chains of thought.

As workflows of this kind become more common and foundation models ingest them, we expect to leave ordering to the model’s planning abilities rather than stack heuristics on heuristics. Technically, each step is transactional, so the agent can execute them in any order. We leave this to future research.

Effectiveness of the Hybrid Approach. A conceptual question is to what extent the agent actually uses WP-generated specs as a baseline, and whether WP is necessary at all — could the agent do all the work, as in [7, 8, 17]? We observe that the WP baseline keeps the agent from suggesting outright wrong specifications, but we lack the data to answer conclusively. A systematic evaluation requires a carefully designed case study, addressing questions such as:

- What is the success rate of the hybrid approach compared to pure LLM inference?
- Does the token cost improve when WP is involved? Or does it get worse because of the additional context?
- Is the set of samples representative? Is the model biased toward the given samples because it already knows the solution (e.g. for standard problems like the `find` function in Sec. II-A)?
- Is the next generation of models changing the picture? The work here has been done with Opus 4.7; what will happen when Opus 5.x hits the market later in 2026?
- How does the approach scale for larger systems? How does it compare to handwritten specs?

We plan to apply inference retrospectively to the Aptos Framework [5], which already has specifications, and compare against the existing ones. We also plan to apply it to a newer in-house project, a complex perpetual trading engine, with results to be reported in future publications.

Skill Design. Writing effective skills is something of a ‘black art’. A few lessons have emerged.

- *Minimize context.* The skill should carry only the information the agent needs — not a tour of the Move ecosystem.

We repeatedly found that extra background dilutes attention, lengthens the working context, and gives the agent room to diverge from the task.

- *Task first, then supporting material bottom-up.* The skill opens with a concise statement of what to do — the task list of Sec. IV — with forward references to the rules and reference material that follow. The body is then ordered bottom-up: spec-language reference first, then tool documentation, then domain-specific guidelines, so that by the time the agent revisits the task list every term has been introduced. With top-down organization, the agent tends to draw conclusions and act on them before reading all the relevant information.
- *Restate the prohibitions.* Agents tend to take shortcuts when searching for a solution. The agent often tries to bypass a hard verification problem by weakening a condition — silencing an abort case, narrowing a postcondition — so the prover reports success. We therefore restate this prohibition at every relevant point in the skill text. Even so, we still occasionally catch CC taking the shortcut; the user-visible task tracker and the verification oracle remain the last line of defense.
- *Testing.* Automated testing of the skill system is hard: the agent’s choices are non-deterministic and coding-agent usage is metered, making CI integration impractical. Our best lever for stabilization is extended manual application.

B. Related Work

Specification inference has a long history. Daikon [6] pioneered *dynamic* invariant detection, learning likely invariants from program traces; subsequent work explored static inference, abstract interpretation, and template-based synthesis, but no unified approach has emerged — tools have relied on heuristics tuned to particular property classes and source languages.

The advent of large language models has revived interest in the area. Pei et al. [7] showed that fine-tuned LLMs can predict program invariants statically at quality competitive with dynamic analysis. Kamath et al. [8] pair an LLM that *proposes* loop invariants with a symbolic checker that *validates* them, retrying on failure — a propose-and-validate loop that recurs in much of the subsequent work. For Verus, AlphaVerus and VeriStruct [18] synthesize Verus-style specifications (and proofs) for Rust functions, modules, and data structures, using fine-tuned LLMs, tree search, or planner-orchestrated phases. For smart contracts in Solidity, PropertyGPT [19] retrieves human-written Certora properties as input examples for specification generation. Most directly related to our work, MSG [17] targets the same Move specification language through a skill-based agentic design that consumes verifier feedback beyond a binary success/failure verdict.

Across these systems the LLM remains responsible for the entire specification; the verifier serves as a binary oracle or, in MSG and our setup, as a feedback channel, but never as a contributor of mechanically derived content.

A separate line of work, exemplified by DafnyPro [20]

and AutoVerus [21], tackles the complementary problem of synthesizing the auxiliary proof annotations a verifier needs when the specification is already given. Our plugin can also synthesize proof hints (Sec. II-B), but this is not the focus of this paper.

Reference elimination, the bytecode transformation underlying our WP analysis (Sec. III), was introduced in [1] and is conceptually shared with Aeneas [13], which compiles Rust to a pure functional model in a target prover (Lean, F*, Coq, HOL4) via forward and backward functions. Aeneas does *not* infer specifications: the user writes them in the target prover, against the translated functional model. We expect our WP approach can be applied similarly to Rust.

Our backend IVL, Boogie [22, 23], also performs WP on the procedural code passed in [11] and passes the result on to the SMT solver. One major difference from our approach is that we aim to generate user-readable conditions, in terms of Move, whereas Boogie’s WP is not intended for external consumption. This is reflected in our mapping of references back to the source language, and in the significant effort put into expression simplification, since raw WP predicates can be very verbose.

In summary, two aspects distinguish our setup from the work above. First, to our knowledge, no prior LLM-based spec-inference system uses a sound mechanical baseline alongside the LLM. WP itself produces a substantial part of the inferred specification — abort conditions, modifies clauses, and consequence-style postconditions — so the AI is left only with what WP cannot recover (loop invariants, high-level conservation laws), and the two streams are validated together by the Move Prover. The AI side runs a propose-observe-refine loop like the systems above; the mechanical baseline narrows the LLM’s search space.

Second, the workflow is delivered as a plugin to a general-purpose, interactive coding agent (Claude Code) via skills, edit-time hooks, and an MCP server, rather than as a custom-built tool: the agent participates in the loop and — most importantly — the user can intervene at any point. To our knowledge, no prior LLM-based inference system for verification languages has this kind of coding-agent integration, including agentic ones such as MSG.

C. Conclusion

We have presented early work on a specification inference tool for the Move Prover that combines a sound, mechanical WP analysis over Move bytecode with an agentic coding CLI; the prover serves as the oracle for both signals. The two inference streams compose in a single instrumentation pipeline, and the workflow is delivered as a Claude Code plugin — skills, hooks, and an MCP server — so inference runs inside the same conversational loop the user already writes Move in. We have applied the system to canonical Move libraries spanning loops, higher-order functions, arithmetic, references, and global state. Future work will stabilize and refine the tool by applying it to the large corpus of safety-relevant Move code used at Aptos for perpetual trading and onchain money management.

REFERENCES

- [1] D. L. Dill, W. Grieskamp, J. Park, S. Qadeer, M. Xu, and J. E. Zhong, “Fast and reliable formal verification of smart contracts with the move prover,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022)*, ser. Lecture Notes in Computer Science, vol. 13243. Springer, 2022, pp. 183–200.
- [2] S. Blackshear, D. L. Dill, S. Qadeer, C. W. Barrett, J. C. Mitchell, O. Padon, and Y. Zohar, “Resources: A safe language abstraction for money,” 2020. [Online]. Available: <https://arxiv.org/abs/2004.05106>
- [3] Aptos Labs, “The Move on Aptos Book,” <https://aptos-labs.github.io/move-book/>, 2023, accessed: 2026-05-09.
- [4] J. Park, T. Zhang, W. Grieskamp, M. Xu, G. D. Giacomo, K. Chen, Y. Lu, and R. Chen, “Securing Aptos Framework with Formal Verification,” in *5th International Workshop on Formal Methods for Blockchains (FMBC 2024)*, ser. Open Access Series in Informatics (OASICS), vol. 118. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 9:1–9:16.
- [5] Aptos Labs, “The Aptos Framework Book,” <https://aptos-labs.github.io/framework-book/>, 2023, accessed: 2026-05-09.
- [6] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin, “Dynamically discovering likely program invariants to support program evolution,” *IEEE Transactions on Software Engineering*, vol. 27, no. 2, pp. 99–123, 2001.
- [7] K. Pei, D. Bieber, K. Shi, C. Sutton, and P. Yin, “Can large language models reason about program invariants?” in *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 27 496–27 520. [Online]. Available: <https://proceedings.mlr.press/v202/pei23a.html>
- [8] A. Kamath, A. Senthilnathan, S. Chakraborty, P. Deligiannis, S. K. Lahiri, A. Lal, A. Rastogi, S. Roy, and R. Sharma, “Finding inductive loop invariants using large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2311.07948>
- [9] Anthropic, “Claude Code,” <https://www.anthropic.com/claude-code>, 2025.
- [10] E. W. Dijkstra, “Guarded commands, nondeterminacy and formal derivation of programs,” *Communications of the ACM*, vol. 18, no. 8, pp. 453–457, 1975.
- [11] M. Barnett and K. R. M. Leino, “Weakest-precondition of unstructured programs,” in *Proceedings of the 6th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*. New York, NY, USA: Association for Computing Machinery, 2005, pp. 82–87.
- [12] Anthropic, “Claude Code Plugins,” <https://docs.claude.com/en/docs/claude-code/plugins>, 2025, accessed: 2026-05-10.
- [13] S. Ho and J. Protzenko, “Aeneas: Rust verification by functional translation,” *Proc. ACM Program. Lang.*, vol. 6, no. ICFP, pp. 116:1–116:31, 2022.
- [14] Aptos Labs, “Move Prover Spec Inference Paper Examples,” https://github.com/aptos-labs/aptos-core/tree/131801f700/third_party/move/move-prover/doc/inference-paper-26/examples, 2026, commit 131801f700, accessed 2026-05-11.
- [15] —, “MoveFlow: AI-assisted Move smart contract development,” <https://github.com/aptos-labs/aptos-core/tree/131801f700/aptos-move/flow>, 2026, commit 131801f700, accessed 2026-05-11.
- [16] W. Grieskamp, T. Zhang, V. Kashyap, and J. Silverman, “Formal verification of imperative first-class functions in Move,” 2026, submitted.
- [17] Y.-F. Fu, M. Xu, and T. Kim, “Agentic specification generator for Move programs,” in *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE 2025)*, 2025, pp. 1286–1298.
- [18] C. Sun, Y. Sun, D. Amrollahi, E. Zhang, S. K. Lahiri, S. Lu, D. L. Dill, and C. Barrett, “VeriStruct: AI-assisted automated verification of data-structure modules in Verus,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2026)*, ser. Lecture Notes in Computer Science. Springer, 2026, arXiv:2510.25015. [Online]. Available: <https://arxiv.org/abs/2510.25015>
- [19] Y. Liu, Y. Xue, D. Wu, Y. Sun, Y. Li, M. Shi, and Y. Liu, “PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation,” in *Network and Distributed System Security Symposium (NDSS 2025)*, 2025.
- [20] D. Banerjee, O. Bouissou, and S. Zetsche, “DafnyPro: LLM-assisted automated verification for Dafny programs,” 2026, dafny Workshop, POPL 2026. [Online]. Available: <https://arxiv.org/abs/2601.05385>
- [21] C. Yang, X. Li, M. R. H. Misu, J. Yao, W. Cui, Y. Gong, C. Hawblitzel, S. K. Lahiri, J. R. Lorch, S. Lu, F. Yang, Z. Zhou, and S. Lu, “AutoVerus: Automated proof generation for Rust code,” *Proc. ACM Program. Lang.*, vol. 9, no. OOPSLA2, 2025.
- [22] M. Barnett, B.-Y. E. Chang, R. DeLine, B. Jacobs, and K. R. M. Leino, “Boogie: A modular reusable verifier for object-oriented programs,” in *International Symposium on Formal Methods for Components and Objects*. Springer, 2005, pp. 364–387.
- [23] K. R. M. Leino and P. Rümmer, “A polymorphic intermediate verification language: Design and logical encoding,” in *TACAS, J. Esparza and R. Majumdar, Eds.* Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 312–327.